AegisX Zero Trust Security Platform

Complexity Simplified, Security Amplified

Technical Project Documentation

Project	Zero Trust Architecture on GCP
Team	AegisX Security Team
Platform	Google Cloud Platform
Status	Deployed & Operational
Date	April 2026
Version	1.0

1. What Is Zero Trust?

Zero Trust is a modern security model built on a simple but powerful principle:

> ***"Never Trust, Always Verify."***

Traditional security models assumed that everything inside a network perimeter was safe. Zero Trust throws that assumption away entirely. Instead, it requires every user, device, and request to be verified — every single time — regardless of where they come from.

Why Zero Trust Matters

- Employees work from anywhere — offices, homes, cafes. VPNs alone cannot secure this.
- Cloud services are accessible from the public internet by default.
- Breaches often come from inside — compromised accounts, stolen credentials.
- Traditional firewalls protect the border but not internal lateral movement.

The Three Core Principles

Verify Explicitly	Always authenticate and authorize based on all available data: identity, location, device, service, workload.
Use Least Privilege	Limit user and system access to only what is needed for the task. Nothing more.
Assume Breach	Design as if the network is already compromised. Minimize blast radius, segment access, encrypt everything.

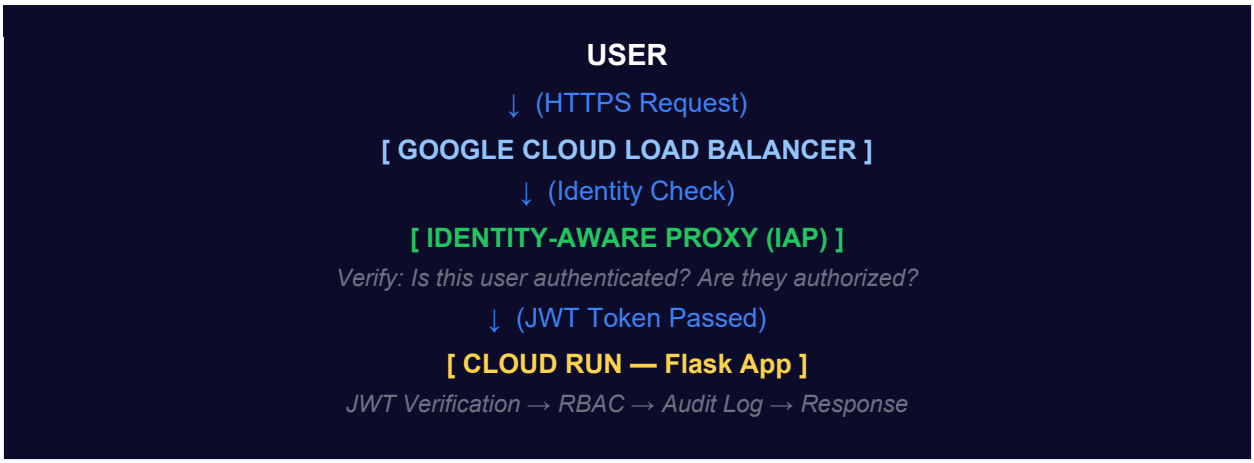
2. Project Overview

AegisX built a Zero Trust internal portal deployed on Google Cloud Platform (GCP). This project demonstrates how to protect a web application using identity-based access control, without relying on VPNs, IP whitelisting, or network perimeter trust.

What We Built

- A Flask web application (Python) with role-based dashboards
- Deployed on Google Cloud Run — serverless, scalable, containerized
- Protected by Google Identity-Aware Proxy (IAP) — forces authentication
- Secured by Google IAM — controls who can access what
- Fronted by a Google Cloud Load Balancer — the single secure entry point
- JWT token verification — the app cryptographically validates every request
- Full audit logging via Google Cloud Logging

Architecture Diagram



3. Components Explained

3.1 Google Cloud Run

Cloud Run is a fully managed serverless platform that runs containerized applications. Our Flask app runs here. It scales automatically from zero to many instances and costs nothing when idle.

- No servers to manage
- Container-based — portable and reproducible
- Direct URL is blocked from public access — traffic MUST come through the Load Balancer

3.2 Google Cloud Load Balancer

The Load Balancer is the only public entry point to the application. All user traffic flows through it. It sits in front of IAP and forwards verified requests to Cloud Run.

- IP Address: 35.244.186.19
- Protocol: HTTPS (port 443) with self-signed SSL certificate
- Without the Load Balancer, IAP cannot be properly enforced on Cloud Run

3.3 Identity-Aware Proxy (IAP)

IAP is Google's Zero Trust gateway. It intercepts every request before it reaches the application and enforces authentication and authorization.

- Forces Google OAuth login for all users
- Checks if the user is on the authorized list (IAM policy)
- Unauthorized users see 'You don't have access' — they never reach the app
- Injects a signed JWT token into every forwarded request

3.4 IAM (Identity and Access Management)

IAM controls who is allowed through IAP. Only users explicitly granted the roles/iap.httpsResourceAccessor role can access the application.

User	App Role	IAP Permission
abdullahammar2991@gmail.com	Admin	httpsResourceAccessor
samararisha12366@gmail.com	Viewer	httpsResourceAccessor

3.5 Flask Application (app.py)

The Flask app is the internal portal that users see after authentication. It has multiple layers of security built in:

- JWT Verification: Cryptographically verifies the IAP token on every request
- RBAC: Checks user role before allowing access to each route
- Audit Logging: Logs every action with user, timestamp, IP, and resource
- Direct Access Protection: Returns 401 if a request arrives without IAP token

Application Routes

```
GET /          → Dashboard (requires IAP + valid JWT)
GET /api/data  → API endpoint (requires IAP + valid JWT)
GET /admin     → Admin panel (requires IAP + admin role)
GET /health    → Health check (public — for Load Balancer probes)
```

4. How It Works — Step by Step

When a user visits https://35.244.186.19.nip.io, here is exactly what happens:

Step	Who Acts	What Happens
1	**Browser**	User types the URL. HTTPS request goes to the Load Balancer IP.
2	**Load Balancer**	Receives the HTTPS request and forwards it to IAP for verification.
3	**IAP**	Checks: Is there a valid Google session cookie? If not, redirect to Google Login page.
4	**Google OAuth**	User signs in with their Google account. Google returns an identity token.
5	**IAP**	Checks IAM policy: Is this user in the authorized list? If not, show 'You don't have access'.
6	**IAP**	User is authorized! IAP signs a JWT token with the user's email and ID, and forwards request to Cloud Run.
7	**Flask App**	Receives request. Extracts the JWT from the X-Goog-IAP-JWT-Assertion header.
8	**Flask App**	Verifies the JWT signature using Google's public keys. Rejects if invalid.
9	**Flask App**	Checks the user's role (admin or viewer) for the requested route.
10	**Flask App**	Logs the action to Cloud Logging. Returns the appropriate response to the user.

5. Security Verification Results

The following tests were performed to validate that the Zero Trust model is working correctly:

Component	Status	Details
Flask App deployed on Cloud Run	✓ PASS	Revision 00005
Static IP reserved (Load Balancer)	✓ PASS	35.244.186.19
Serverless NEG connected to backend	✓ PASS	us-central1
Load Balancer forwarding HTTPS (443)	✓ PASS	Self-signed cert
IAP enabled on backend service	✓ PASS	Enforced
Unauthorized user blocked by IAP	✓ PASS	basselbedan@gmail.com denied
Authorized user (admin) can access dashboard	✓ PASS	abdullahammar2991
Authorized user (viewer) can access dashboard	✓ PASS	samararisha12366
Viewer cannot access /admin route	✓ PASS	403 Forbidden returned
Direct Cloud Run URL returns 401	✓ PASS	Bypassing LB blocked
IAP JWT cryptographically verified	✓ PASS	ES256 algorithm
Audit logging active	✓ PASS	Cloud Logging

What Each Test Proves

- **Zero Trust Gate:** Unauthorized user blocked

  Users NOT in the IAM policy cannot access the app at all. IAP stops them before they even reach Flask.
- **Least Privilege:** Viewer cannot access /admin

  Even authenticated, authorized users are limited to what their role permits.
- **Bypass Prevention:** Direct URL returns 401

  There is no way to skip IAP. The Cloud Run URL itself enforces authentication.
- **Token Integrity:** JWT cryptographically verified

  The app never trusts headers blindly — it verifies every token's cryptographic signature.

6. How to Add New Users

Adding a new user requires two steps. Both are required — skipping one means the user either cannot get through IAP, or can get through but has no role assigned.

Step 1 — Grant IAP Access (Cloud Shell)

Run this command in GCP Cloud Shell:

```
gcloud iap web add-iam-policy-binding \
  --resource-type=backend-services \
  --service=zerotrust-backend \
  --member="user:newuser@gmail.com" \
  --role="roles/iap.httpsResourceAccessor"
```

Step 2 — Assign App Role (Edit app.py)

Open the app.py file and add the user to the ROLES dictionary:

```
ROLES = {
    "abdullahammar2991@gmail.com": "admin",
    "samararisha12366@gmail.com":  "viewer",
    "newuser@gmail.com":           "admin",  # or "viewer"
}
```

Step 3 — Redeploy the App

```
cd ~/zerotrust-app
gcloud run deploy zerotrust-app \
  --source . \
  --region=us-central1 \
  --no-allow-unauthenticated
```

Role	Can Access	Cannot Access
admin	/, /api/data, /admin, /health	Nothing
viewer	/, /api/data, /health	/admin (403 Forbidden)

7. Technology Stack

Layer	Technology	Purpose
Application	Python Flask 3.0	Web framework for the internal portal
Server	Gunicorn	Production WSGI server (replaces dev Flask server)
Container	Docker	Packages the app for consistent deployment
Runtime	Google Cloud Run	Serverless container execution
Security Gateway	Google IAP	Authentication and authorization proxy
Access Control	Google IAM	User permission management
Networking	Cloud Load Balancer	Single HTTPS entry point
Token Security	PyJWT + cryptography	JWT signature verification
Monitoring	Cloud Logging	Audit trail for all access
SSL	Self-signed cert	HTTPS encryption (demo environment)

8. Quick Reference

Access URLs

Portal (LB)	https://35.244.186.19.nip.io
API Endpoint	https://35.244.186.19.nip.io/api/data
Admin Panel	https://35.244.186.19.nip.io/admin
Health Check	https://35.244.186.19.nip.io/health
Cloud Run URL	https://zerotrust-app-848617071252.us-central1.run.app (blocked — 401)

GCP Project Details

Project ID	aegisx-zero-trust-demo
Project Number	848617071252
Region	us-central1
Backend Service	zerotrust-backend (ID: 7799598352765165470)

Key Commands

View Logs

```
gcloud run services logs read zerotrust-app --region=us-central1
```

Redeploy App

```
cd ~/zerotrust-app
gcloud run deploy zerotrust-app --source . --region=us-central1 --no-allow-
unauthenticated
```

Add Authorized User

```
gcloud iap web add-iam-policy-binding \
  --resource-type=backend-services \
  --service=zerotrust-backend \
  --member="user:email@gmail.com" \
  --role="roles/iap.httpsResourceAccessor"
```